

Generating Test Cases from Requirements: A Case Study in Railway Control System Domain

Hanyue Zheng, Jincao Feng, Weikai Miao, Geguang Pu

Software Engineering Institute

East China Normal University

Shanghai, China

zhenghanyue813@gmail.com, jincaofeng@foxmail.com, {wkmiao, ggpu}@sei.ecnu.edu.cn

Abstract—Requirements-based testing is one of the most commonly used ways to ensure the correctness of software, especially for embedded control software in safety-critical domains such as spacecraft and railway systems. Many industrial standards such as the DO-333 and EN50128 also request rigorous requirements-based software testing. To test embedded control software effectively and efficiently, generating high-quality test cases automatically is extremely important. However, existing methods for generating test cases from requirements require intensive manual efforts and expertise. To address this problem, we proposed an automatic requirements-based software testing method for embedded control software. To obtain automatic test case generation and precise test oracles derivation, requirements specification should be precise and readable for the industrial practitioners. Therefore, we use the light-weight domain-specific formal description language, CASDL (Casco Accurate Specification Description Language) for the industrial practitioners to define software requirements into formal specifications at the first step. Based on the formal specification, we propose an algorithm to automatically generate test inputs that satisfy the MC/DC criteria suggested by typical industrial standards and precise test oracles can be derived by “running” the specification with such test inputs. To this end, we proposed an algorithm for simulating the formal specification to generate the test oracles, *i.e.*, the expected outputs corresponding to the test inputs. To facilitate the application of this method in the industry, we have built a tool that can automatically perform the overall testing process. To validate and evaluate its effectiveness in real industrial projects, we have applied it in testing a real Automatic Train Protection (ATP) system provided by our industrial partner, the Casco Signal Co., Ltd (one of the largest railway control system companies in China). In the case study on ATP requirements, our approach generated test cases for 129 requirement items following MC/DC criteria and caught 40 inconsistencies between Casco’s requirements and implementation.

I. INTRODUCTION

Software testing is a common and widely-used approach to ensure the reliability of software, especially for embedded control software in the industry used in safety-critical areas. Faults in these systems may cause heavy losses than other systems. High-quality software testing relies on high-quality test cases. Therefore, high-quality test cases play an important role in finding faults in these industrial embedded control systems. However, producing high-quality test cases requires intensive manual efforts and expertise. Thus, generating high-quality test cases automatically is a promising direction, though it

is difficult. There are two challenges: *how to generate high coverage test inputs* and *how to obtain the corresponding test oracles*. Chen et al. proposed metamorphic testing [1] to further test the software based on existing selection criteria. Although metamorphic testing can obtain the oracles by construction, it has no idea how to improve the coverage of the software. Symbolic execution [2] is another common way to generate test inputs for software. Because of the advantage of symbolic execution, it can reach high coverage. However, it cannot obtain the test oracles.

To address the above challenges, one promising way is to generate test cases from requirements specifications, which can achieve high coverage with known test oracles. However, requirements are usually written in natural language, which makes understanding them automatically become challenging. Due to the ambiguity of natural language, it is difficult to capture the semantic of the requirements for tools. Although researchers spare no effort to translate the natural language to other forms to understand its meaning [3]–[6], the work of understanding requirements written in natural language is far away from satisfying. Fortunately, the requirements in the industrial control system domain are more structured and formal than others, which brings out the possibility of automated test case generation. To ensure the test cases are qualified for embedded control software testing, some criteria have been proposed in many industry standards, *e.g.*, DO-333 [7] suggests the test cases generated from requirement should follow MC/DC criteria [8]. IEC-61508, ISO-26262, and EN-50128 also require rigorous requirements-based testing

However, writing test inputs that follow MC/DC criteria manually can be time-consuming. According to the experience of the engineers in our partner company, Casco Signal Co., Ltd., it will take around two months for one engineer to write test inputs that satisfy MC/DC criterion for one project. Obviously, it is a huge workload. Consequently, many researchers and engineers are trying to automatically generate test cases with high quality and high coverage. However, there are two main challenges while generating test cases automatically. Firstly, it is difficult to understand the requirements written in natural language automatically. It is due to the limitation of the natural language processing techniques and the innate ambiguity of the natural language. Secondly, it is difficult to extract conditions and generate test cases that follow MC/DC

✉ Weikai Miao is the corresponding author.

criteria for complex requirements. It is because of the lack of supporting tools in the industry.

To solve the above challenges, in this paper, we proposed a method to generate test cases from the requirements and evaluate them by a case study in the railway control system domain. A brief idea of our method is: First, we use the lightweight domain-specific formal description language CASDL (Casco Accurate Specification Description Language), which was proposed in our previous work [9], to define software requirements into the formal specification. Based on the formal specification, we proposed a test inputs generation algorithm to automatically generate test inputs that satisfy the MC/DC criteria suggested by typical industrial standards. Since the specification is formal, precise test oracles can be derived by “running” the specification. To this end, we proposed an algorithm for simulating the formal specification to generate the test oracles, *i.e.*, the expected outputs corresponding to the test inputs. Then, we can test the implementations of the requirements using the test inputs, and comparing the outputs of the implementations and the test oracles, to find out the inconsistencies between the implementations and requirements. In the evaluation, we found 40 inconsistencies between our partner company’s implementations and requirements. By looking into the inconsistencies, we found some representative issues in either requirements or implementations, which shows the effectiveness of our method. Our partner company appreciated our work for helping them generate test cases automatically and finding the potential issues. In summary, the main contributions of this paper are as follows:

- We proposed a method for generating test cases from requirements that follow the MC/DC criteria suggested by DO-333 and EN-50128. The test cases can be used to validate the implementations of the system.
- We implemented a framework that can automatically generate the test cases that follow the MC/DC criteria from the CASDL specification.
- We evaluated the framework on the requirements of the ATP system provided by our partner company Casco Signal. In the evaluation, we generated test cases for 129 requirement items and found 40 inconsistencies.
- By generating the test cases following the MC/DC criteria, we help our partner company Casco Signal satisfy the standard of EN-50128.

The organization of this paper is as follows: In Section II, we give a specific example to describe the process of generating test cases from requirements in natural language. In Section III, we demonstrate the method for test case generation in detail and give the specific algorithm for MC/DC test case generation. In Section IV, we exhibit the experiment results and sufficiently analyze the inconsistencies found in the experiment. In Section V, we introduce the related work both in test case generation and formal language. Finally, in Section VI, we summarize the contribution of our method and propose the promising research direction in future work.

TABLE I
NATURAL LANGUAGE VERSION OF REQUIREMENT FRAGMENT.

When initialization or the message has expired, set all variables as default value. When a new message available, set the corresponding value from the new message. Otherwise, remain unchanged.
--

II. ILLUSTRATIVE EXAMPLE

The railway control system is designed to ensure the trains run normally and securely. It consists of multiple subsystems, such as a signal control system and the interlocking system. In most situations, the system runs periodically for a certain purpose with initial values and environment settings. The information like the values of variables can be collected during the execution, the state of the system can also be changed, and the task being run is usually computation-oriented.

Almost all the requirements in the railway control system domain are written in natural language. However, the description of natural language can be ambiguous, which makes it confusing to understand. Since the correctness of requirements is especially important, natural language is not the best choice for requirements writing. To solve this problem, we make some changes to the formal language proposed by Wang *et al.* [10] to suit the situation in the railway control system domain and analyzing on the rewritten requirements to avoid problems produced by ambiguity.

As shown in Fig 1, the workflow of our method has five steps. ① Rewriting the natural language requirements into formal specifications with CASDL. ② Extracting constraints based on the previous formal specification. ③ Generating MC/DC criteria-based test inputs with the constraints. ④ Obtaining test oracles by simulating the formal specification with the test inputs and other related information. ⑤ Combining the test inputs and test oracles to generate test cases. We then give a brief step-by-step example to show this process.

① Table I shows the natural language version of this example. The requirements fragment describes how to parse and store information received in the ATP system (a subsystem of the railway control system). The system handles the messages differently according to different situations. When all the conditions are not satisfied, the system remains unchanged. The natural language version of this requirements fragment is not clear enough and programmers can be confused if they are not an expert in the railway control system domain. For example, the description *when a new message available* can be fulfilled in many situations, such as the message is not expired, the message is in a certain form or a concerned variable holds a certain value. To describe the requirements with more accuracy, we need to translate them into the CASDL version shown in Fig. 2.

② In the CASDL version (Fig. 2), the natural language requirements fragment is written as a procedure named *Other-ATP(k)*, it parses and stores information received at *kth* cycle. The procedure is composed of a conditional sen-

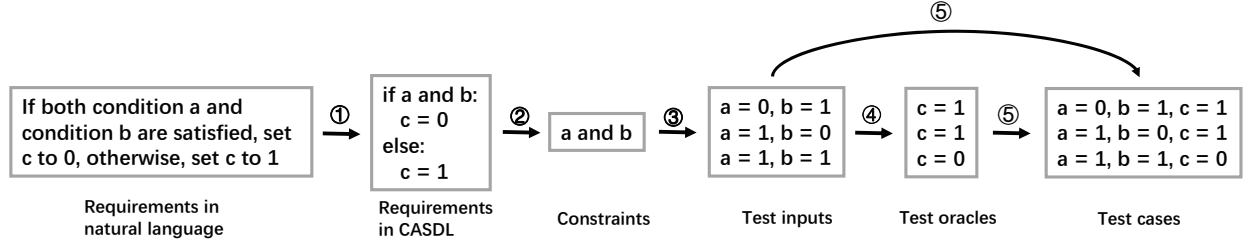


Fig. 1. Workflow of this method.

```

def OtherATP(k):
    if (Initialization
        or (not OtherATP.messageValid(k))):
        OtherATP.BeaconId = None
        OtherATP.EnableDoorOpening_A = False
        OtherATP.LatestTimeOtherCore = INVALID_LOOP_HOUR
    elif (OtherATP.messageAvailable(k)):
        OtherATP.BeaconId =
        OtherCCsynchroReport.BeaconId
        OtherATP.EnableDoorOpening_A =
        OtherCCsynchroReport.EnableDoorOpening_A
        OtherATP.LatestTimeOtherCore =
        OtherCCsynchroReport.LatestTimeOtherCore(k)
    else:
        pass
    return OtherATP
  
```

Fig. 2. CASDL version of requirement fragment.

tence. *Initialisation* in the first branch condition represents the initial state described in natural language requirement, and *OtherATP.messageValid(k)* shows whether the message received at *kth* cycle is expired. If the message is expired, system sets the value as **False**, otherwise, it is set to **True**. To solve the ambiguity described in the previous paragraph, we asked experts in railway domain and use variable *OtherATP.messageAvailable(k)* to represent the sentence *When new message is available*. *OtherATP.messageAvailable(k)* describes the effectiveness of message at *kth* cycle. In this function, the system queries whether there is a redundant ATP message and takes different measures under different situations. *OtherCCsynchroReport* collects the details of received message. *BeaconId*, *EnableDoorOpening_A* and *LatestTimeOtherCore(k)* are attributes of *OtherATP*, their values can be changed according to the conditions described in natural language requirements fragment. The default value of them are set to **INVALID_LOOP_HOUR**, which is a constant represents the invalid loop hours, **None** and **False** respectively.

The confusing part in the natural language version is more clear in the CASDL version. *when new message available* means when the value of a boolean variable *OtherATPmessageAvailable* turns to **True** at *kth* cycle. People who know nothing about the railway control system can understand the requirements fragment clearly.

③ After the translation process, on the requirements written in CASDL, we collect the constraints and then obtain test inputs by solving the constraints using the SMT solver. To collect constraints, we extract the conditions that appear

in CASDL. The constraints in Fig. 2 are shown in the second column of Table II. At the step of generating test inputs, we propose an algorithm and choose MC/DC (Modified Condition/Decision Coverage) criteria suggested by DO-333 standards. The test cases that follow MC/DC criteria require each atomic condition in a boolean expression should affect the value of the expression independently, which means when one of the atomic conditions changes its truth value, the value of the expression should also change.

The test inputs generated in this illustrative example are shown in the third column of Table II. For the constraint *Initialization or (not OtherATPmessageValid(k))*, the first three test inputs fulfill MC/DC criteria, since in these test inputs, both *Initialization* and *OtherATPmessageValid(k)* can affect the truth value of the expression. In the first and third test inputs, the value of *Initialization* is opposite while the value of *OtherATPmessageValid(k)* stays the same, and the truth value of the expression changes in the two test inputs. The same situation happens in the second and third test inputs, the difference is that *Initialization* stays the same value and *OtherATPmessageValid* changes its value. As for the last two test inputs, the expression's value changes when *OtherATPmessageAvailable(k)* changes its truth value.

④ To construct complete test cases, we need to obtain test oracles corresponding to the previous test inputs. With the test inputs and other essential information, we are able to perform the simulation on the formal specification to obtain the test oracles. The results of the test oracles generation are shown in the last column of Table II.

⑤ After combining the test inputs and test oracles together, we can use the test cases to validate the implementation of function **OtherATP** in the development of the software.

III. APPROACH

A. From Natural Language to CASDL

Since the gap between natural language and formal language exists, we have to translate natural language requirements into the formal language version, which is represented by a template specially designed for the requirements. It is composed of *Block*, *DefBlock* and *Tasks*.

- *Block* is used to identify the state change of a variable that represents the state of the system.
- *DefBlock* shows the assignment or value change of a normal variable in a computation task.

TABLE II
CONDITION CONSTRAINTS, TEST INPUTS, AND TEST ORACLES.

ID	Condition Constraints	Test Inputs	Test Oracles
1	Initialization or (not OtherATPmessageValid(k))	Initialization = true; OtherATPmessageValid(k) = true;	OtherATP.LatestTimeOtherCore = INVALID_LOOP_HOUR; OtherATP.BeaconId = None; OtherATP.EnableDoorOpening_A = False;
2	Initialization or (not OtherATPmessageValid(k))	Initialization = false; OtherATPmessageValid(k) = false;	OtherATP.LatestTimeOtherCore = INVALID_LOOP_HOUR; OtherATP.BeaconId = None; OtherATP.EnableDoorOpening_A = False;
3	not (Initialization or (not OtherATPmessageValid(k)))	Initialization = false; OtherATPmessageValid(k) = true;	OtherATP.LatestTimeOtherCore = OtherCCsynchroReport.LatestTimeOtherCore(k); OtherATP.BeaconId = OtherCCsynchroReport.BeaconId; OtherATP.EnableDoorOpening_A = OtherCCsynchroReport.EnableDoorOpening_A;
4	OtherATPmessageAvailable(k)	Initialization = false; OtherATPmessageValid(k) = true; OtherATPmessageAvailable(k) = true;	OtherATP.LatestTimeOtherCore = OtherCCsynchroReport.LatestTimeOtherCore(k); OtherATP.BeaconId = OtherCCsynchroReport.BeaconId; OtherATP.EnableDoorOpening_A = OtherCCsynchroReport.EnableDoorOpening_A;
5	not (OtherATPmessageAvailable(k))	Initialization = false; OtherATPmessageValid(k) = true; OtherATPmessageAvailable(k) = false;	NULL

- *Tasks* usually describe the detailed computation process with a control flow graph.

There is also a dictionary that stores the type and value for each variable that appeared in the requirements.

When translating the natural language version of requirements into the formal language version, engineers should know the physical meaning of natural language requirements clearly, then define the inputs, outputs, and data structure stipulated by the template by themselves. Data dictionary also be structured during this process when collecting all the variables which do not appear in the dictionary. After that, the requirements in CASDL can be extracted automatically. We compile each part of the requirements into a structured model which is also composed of the parts in formal language. We then analyze the model to generate test cases, which will be introduced in the next two subsections.

B. Generating Test inputs from CASDL

Test cases are composed of test inputs and test oracles. In our method, we firstly generate test inputs and then get test oracles based on the test inputs we have, and finally combine them to get test cases. We then introduce how to generate test inputs following MC/DC criteria for CASDL.

MC/DC is a strict criterion and is widely used in software testing and verification, especially in safety-critical areas. As one of the safety-critical areas, this criterion is also requested in the railway domain. As stipulated by DO-333, test cases should satisfy MC/DC criteria. To meet the request of DO-333, Algorithm 1 is designed by us to generate test cases that satisfy MC/DC criteria. *ConExpr* in the algorithm is the condition expression in CASDL. A *ConExpr* can be defined as follows:

- *ConExpr* ::= *ConExpr* *Symbol* *ConExpr* | *Const* | *Var*
- *Symbol* ::= > | < | = | + | - | * | / | and | or

where *Const* and *Var* represent constant value and variable respectively. *Constraints* is the output of the function **MCDC** of Algorithm 1 and the value of which incrementally iterate

in the execution of the algorithm. The system executes this algorithm repeatedly according to the type of the symbol until the condition expression becomes atomic.

When the recursion meets the atomic expression, we call an SMT solver to generate test inputs from constraints. SMT solver is used to solving the satisfaction problem of the first-order formula. If the formula can be satisfied in certain circumstances, the solver returns an assignment for each variable in the formula that makes the formula become true. The solving results of the SMT solver for the constraints will be one of the test inputs following MC/DC criteria. If the SMT solver can solve the constraints, we will add the solving results into the test suite. Finally, in the function **TIG** of Algorithm 1, SMT solver returns *Tests*, which is the group of the test inputs towards MC/DC criteria. Due to the limitation of the existing SMT solvers, they may not be able to solve all constraints collected in the algorithm, thus, the requirement coverage would not be 100% after running the algorithm.

C. Generating Test oracles by simulation

We assume that if the implementation of code is correct, then the execution result of the code should be consistent with the requirements. Thus, we propose to do a simulation on the CASDL version requirements with the test inputs in the previous subsection to achieve test oracles. As shown in Figure 3, it is the workflow of the simulation. With inputs shown in the diagram, the simulation is executed inside a simulator and finally gets the corresponding test oracles.

The inputs of the simulation process are composed of 4 parts. The test inputs follow the MC/DC criteria, and the combination of test inputs and the corresponding outputs of the simulation are test cases. Requirement items are the subjects to be executed in the simulator, the other inputs provide the conditions and circumstances for simulating the requirement items. The requirement items at this step are written in CASDL. Environment settings is a list that stores the

Algorithm 1 Test Inputs Generation (TIG) Algorithm

Input: ConExpr
Output: Tests

```

1: function TIG(ConExpr):
2:   Tests ← ∅
3:   Constraints ← True
4:   Tests ← MCDC (ConExpr, Constraints, Tests)
5:   return Tests
6:
7: function MCDC(ConExpr, Constraints, Tests):
8:   if ConExpr is an atomic expression then
9:     test = SMT_solver (Constraints)
10:    if test ≠ null then
11:      return Tests.add (test)
12:    else
13:      return Tests
14:   else
15:     leftExpr ← ConExpr.left
16:     rightExpr ← ConExpr.right
17:     if ConExpr.symbol = Symbol.and then
18:       Tests ← MCDC (leftExpr, (Constraints & rightExpr), Tests)
19:       Tests ← MCDC (rightExpr, (Constraints & leftExpr), Tests)
20:     else if ConExpr.symbol = Symbol.or then
21:       Tests ← MCDC (leftExpr, (Constraints & !rightExpr), Tests)
22:       Tests ← MCDC (rightExpr, (Constraints & !leftExpr), Tests)
23:   return Tests

```

TABLE III
EXAMPLE OF DATA DICTIONARY.

Variable name	Variable type	Critical	Variable value
AxleLockedDetectionAvailable	BOOLEAN	YES	TRUE

environment information related to the simulation. It contains various environment parameters as well as their values. Such as current temperature, allowed acceleration. And dictionary collects all variables that have appeared in the requirement and their values. The dictionary is composed of 4 parts, variable name, variable type, whether it is a critical variable and variable value. An example is shown in Table III.

The tasks executed during simulation are usually computation-oriented, so they can be represented by control flow graphs. At the beginning of a simulation process, the system checks the preconditions stipulated by requirements first. the preconditions can be a Boolean expression, and the value of each condition item can be calculated from the test inputs, environment parameters, and data dictionary. Every change of the variables will be recorded in the dictionary during the simulation.

The simulation process may last more than one cycle, so at the end of the execution at each cycle, the system updates the values in the data dictionary and check whether it is the last cycle of the simulation process. If not, the system updates the value of the cycle clock and starts a new cycle of simulation with the updated dictionary, and environment parameters used in the new cycle. The final output of the simulation is the test oracles corresponding to the test inputs.

IV. EMPIRICAL EVALUATION

A. Evaluation Setting

1) *Hardware:* The case study was run on a 64bit Microsoft Windows 10 operating system with 16GB RAM and processor

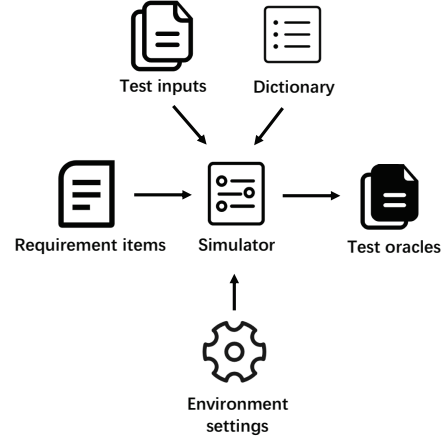


Fig. 3. Simulation process.

3.40GHz Intel (R) Core (TM) i7-6700 CPU.

2) *Implementation:* The back end of CASDL is implemented with C# and the front end is implemented with Javascript, CSS, and Html. The syntactic parser used by us is ANTLR [11]. We also make the graphic interface more friendly to users. In the experiment, we choose Z3 as the SMT solver to generate test inputs from constraints. Z3 [12] is a powerful SMT solver developed by Microsoft to solve satisfaction problem of first-order formula.

3) *Background:* Aimed at ensuring the reliability of our method and tool, we applied our test case generation method on Casco's automatic train protection (ATP) signal requirements and illustrate a case study in this paper.

The protection system described by ATP requirements is a computing-oriented embedded periodic system. It contains computation tasks in each cycle of the execution. The computation tasks can be divided into two categories: *DefBlock* and *Block*. *DefBlock* illustrates the detailed computation of a function and the *Block* mainly describes the state transition of a state machine. There are also some external environment parameters included in the calculation.

The ATP requirements document used in the case study contains 129 requirement items. The requirement is rewritten in CASDL before analysis.

B. Evaluation results

1) *Requirement Coverage:* Coverage is an important indicator for evaluating the quality of the test cases. As requested by DO-333, the railway security-related principle requirements should satisfy MC/DC criteria. Thus we calculate the coverage of MC/DC with the following formula $\frac{m}{M}$, where M represents the number of MC/DC test inputs that should be generated in theory and m represents the number of test inputs which actually solved by Z3 in the experiment.

Using the formula above, the coverage of the MC/DC test cases generated by our method is 92.89%. It is a pretty high coverage for test inputs in requirements, and the engineers in Casco significantly satisfied with the test cases generated

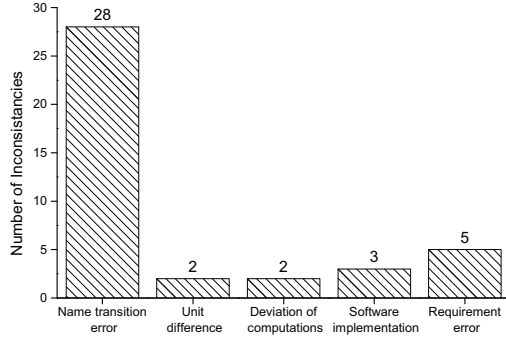


Fig. 4. Consistency analysis result.

by us. Obviously, the uncovered parts of the requirements are the constraints that can not be solved by Z3. There are two situations that may lead to this result. One is that the requirement fragments are actually inaccessible. The other is that the Z3 solver is not strong enough to solve the constraints. In the further investigation, we find that the requirement has some unreachable paths which be taken as unsatisfiable by Z3.

2) *Consistency Analysis*: Intuitively, if the code implementation of the software is consistent with the requirement, then the test cases generated from requirements could be directly used in software development. Here we apply the test cases in the testing procedure of the development and find errors both in requirements and implementation.

In the experiment, a total of 40 inconsistencies are revealed after 15273 cycles' execution. As shown in Figure 4, we categorised the inconsistencies into five types. 28 of them are name transition errors, which means the same variable has different names in the requirement and implementation. Two inconsistencies are unit difference, which occurs when a variable has different dimension (e.g., millisecond and second) in requirements and implementation. Another two inconsistencies are due to the deviation of computations, which due to the different computational accuracy used between requirement simulation and code execution. There are also three and five errors found in the software implementation and requirement respectively. More details of these inconsistencies will be illustrated in the next subsection.

C. Issue examples

Here we take some classical issues as examples to illustrate two of the five inconsistency types (i.e., name transition error, unit difference, deviation of computations, software implementation, requirement error) mentioned above.

1) *Unit difference*: The requirement item in Fig. 5 describes the process of computing instantaneous acceleration of wheels when the wheel motion and acceleration are measurable. It encountered an inconsistency when runs to the 34th cycle. The correct value of *InstantaneousWheelAcceleration* should be 675000 while the code returns a 675.

The study on this inconsistency shows that the error appears when the unit is different in requirement items and imple-

```
def InstantaneousWheelAcceleration(k):
    return ((abs(TeethCounter(k)
        - TeethCounter(k-2))
        - abs(TeethCounter(k-2)
        - TeethCounter(k-4)))
        * MaxCogCalibration(k-1)
        / pow(2*ATP_CYCLE_TIME))
```

Fig. 5. Issue example related to the incorrect computational result.

```
def SlippingEnded(k):
    if (MotionUnderEstimationState(k-1)
        is not SLIPPING):
        slipping_ended_counter = 0
        return False
    elif (FilteredWheelAcceleration(k)
        < ATPsetting.SlippingStopAcc
        and FilteredWheelAcceleration(k)
        > ATPsetting.SlidingStopAcc):
        slipping_ended_counter =
        slipping_ended_counter + 1
        return (slipping_ended_counter
            >= ATPsetting.SlippingGripRecoveryTime)
    else:
```

Fig. 6. Issue example related to the incomplete requirement.

mentation code. In this issue, the unit in the requirement is millisecond (ms) while in the code, the unit is second (s). We found this error in the 34th cycle because the system does not meet the conditions entering a target state in first 30 cycles.

Units are frequently used in the calculation of specific scenes, especially in some safety-critical areas like aviation and railway control systems. Since the computation results have a great impact on the safety of the system, it is important to keep the consistency of the unit of measurement. The requirement fragment in this example does not point out what exactly the unit is, which causes ambiguity in implementation. Therefore, requirements must label the unit explicitly when describing the computation-related requirement fragments.

2) *Requirement error*: Another issue found during the execution is caused by incomplete requirements. The requirement fragment is shown in Fig. 6. This requirement fragment describes one of the conditions for judging whether to end the idling state. The error appeared in the first cycle when the requirement returns **NULL** and the program returns 0. In this issue, the requirement is not complete. It does not regulate what will happen in the *else* statement. In this situation, the requirement returns **NULL** and the program returns the default value, which makes the inconsistency.

Inconsistency might happen when there are no clear instructions in requirements, and this can lead to uncontrollable programs. The inconsistency situation happened in this requirement fragment because it does not regulate what will happen when all the conditions are not met. Therefore, well-described requirements should consider all situations that could happen in the system, and explicitly describe the tasks under all the situations, or errors may happen under certain circumstances.

D. Discussion

There are three points that deserved discussion.

1) *Summary of the evaluation:* In the evaluation on ATP requirements, our method and tool obtained a high coverage of 92.89%, which far beyond the expectation of the engineers in Casco. They thanked us for helping them realize automated test case generation, which greatly reduced their labor costs and work time. On the other hand, the 40 inconsistencies founded by us also provided guidance on further refinement of the code and requirements. In safety-critical areas such as railway and aviation, any tiny errors can not be ignored. The unit difference and incomplete requirements introduced in the issue study are such errors that will lead to disastrous consequences. The error report generated by our tool summarized so many critical errors on Casco's fully-tested requirements, which sufficiently demonstrate the effectiveness of our method and save the ATP system of them.

2) *Advantage of our method:* In traditional practice, automatically generating test cases from code is a regular choice. It is unimaginative to automatically generate test cases from such natural language requirements for that the requirements are imprecise and ambiguous. They always employ a lot of manpower to generate test cases by hand, which is labor and toil with mind and body. We skillfully transform the requirement into a precise formal specification, which can be analyzed and validated exhaustively. Thus the hard problem of automatically generating test cases from requirements has been solved by us in a sense.

3) *Limitations and future work:* Although our method has so many advantages, we also explore its limitations and hope to solve these hard problems in future works. On one hand, although CASDL is suitable for railway system and is easy to learn for engineers. The time cost for rewriting requirements in natural language to CASDL specification is great, which takes 80% of the time in generating test cases. Therefore, applying natural language processing technology directly to the requirements and automatically generate test cases from natural language requirements is what we aim to do in the next work. In our opinion, with the breakthrough of natural language in requirements analysis, in the future world, engineers only need to write requirements with natural language and the error-free code is automatically generated. On the other hand, our model is accurately designed for the railway system, which is not universal enough. Thus in future work, we aim to design a more general model for embedded cycle system (*i.e.*, control systems of spacecraft, railway, *etc.*) first and then expand the model to be suitable for other systems such as computer operating systems, application systems.

V. RELATED WORK

Over the past two decades, people have been wishing to make software testing a cost-effective process due to the huge labor cost [13], and some progresses have been made. [14]–[16]. As a measure of software quality, software testing can evaluate whether a system satisfies the expected results by using test cases, and it has been widely used both in

research and in industry. [17]–[19] Generating test cases by hand is time-consuming work, as a consequence, people try to find an automated way to do this work. And many works that focus on automatically generate test cases from different areas have made great progress. Such as generating test cases from requirements [20], [21], android apps [22], [23], web services [24] and various UML diagrams [25], [26] *etc.*. The test cases in requirements-based testing can generated from UML use case [27], by using guarded assertions [28] and from various requirements models [29], [30].

For requirements-based testing, most of the test cases are generated from models. How to build a reliable requirements model is a problem that can not be ignored. Engineers and researchers have made a lot of effort on requirements modeling. One method is modeling requirement by state machine, such as ASM (Abstract State Machine) [31] and StateChart [32]. Simulink [33] which based on StateChart is also widely used in the industry. Besides, some superior tools like DOORS, SCADE [34], RTCASE [35], and PREMA [36] also have been developed for the purpose of requirement modeling.

Coverage describes the degree of the program or a system that has been tested. And coverage criteria are used to judge the quality of the test cases. Classic coverage criteria are statement coverage, branch coverage, decision coverage, condition coverage, modified condition/decision coverage. Statement coverage [37] involves the execution of all the statements at least once in the source code. Branch coverage requires that each branch in the program should have been executed at least once during testing. Decision coverage testing is to cover and validate all the accessible source code by checking and ensuring that each branch of every possible decision point is executed at least once. Condition coverage is used to check individual outcomes for each logical condition. Condition coverage offers better sensitivity to the control flow than decision coverage. MC/DC is stronger than the coverage mentioned above, it requires each condition should influence the result of expression independently at least once. Therefore, generating test cases in safety-critical areas usually uses MC/DC.

Formal language can perfectly avoid the ambiguity of natural language, so it is always considered a reliable way to describe software requirements, especially in industry control areas. Some efforts have been made to propose suitable formal language for modeling. Classic formal language like Z, B, and Event-B can be used to model the requirement. And the B method was successfully applied to the design of the railway system in France. Besides, some domain-specific formal language also performs well in their domains. Such as AADL [38] and AASRD [39] are successfully used in aviation area and SPARDL [10] in embedded control system.

VI. CONCLUSION

In this paper, we proposed a method for automated test case generation from requirements. Firstly, we translate the natural language requirements to the CASDL version. Secondly, we proposed an algorithm that takes advantage of the Z3 solver to generate test inputs that follow the MC/DC criteria from

requirements written in CASDL. Thirdly, with test inputs, test items, data dictionary and environmental parameters in the list, we can get test oracles by performing simulation in a simulator. And finally, we can get the complete test cases by combining the test inputs and test oracles. In order that requirements engineers can use this method more conveniently, we also provide a framework to support this work.

The case study on the requirements of the ATP signal system achieves coverage of 92.89%, the uncovered parts are because the Z3 solver can not solve the constraints. During the consistency analysis, we run the test cases generated from requirements in the implementation code of the ATP system and find 40 inconsistencies in total. 28 of them are name transition errors, 2 of them are due to the unit used in requirements and code is different. And 2 errors are because of the deviation during the computation process. 3 of them are caused by incorrect software implementation and the last 5 errors are caused by the requirement itself.

This work plays an important role in helping the requirement documents in Casco Signal satisfy the EN-50128 standard. The developers and managers in Casco Signal appreciated our work and the issues we found in the evaluation. They fixed the issues we found and have incorporated our method in their development tool-chain. For future work, we will continue collaborating with Casco Signal to improve the quality of their requirement documents and validating the implementation of the safety-critical systems.

ACKNOWLEDGMENT

We thank the anonymous reviewers for their valuable feedback. Weikai Miao is supported by the NSFCs of China No.61872144. Geguang Pu is supported by NSFC Project No. 61632005.

REFERENCES

- [1] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," *arXiv preprint arXiv:2002.12543*, 2020.
- [2] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.
- [3] T. Lei, F. Long, R. Barzilay, and M. Rinard, "From natural language specifications to program input parsers," in *ACL*, 2013, pp. 1294–1303.
- [4] Z. Zhong, J. Guo, W. Yang, J. Peng, T. Xie, J.-G. Lou, T. Liu, and D. Zhang, "Semregex: A semantics-based approach for generating regular expressions from natural language specifications," in *EMNLP*, 2018.
- [5] L. Dong and M. Lapata, "Language to logical form with neural attention," *ACL*, 2016.
- [6] J.-U. Park, S.-K. Ko, M. Cagnetta, and Y.-S. Han, "Softregex: Generating regex from natural language descriptions using softened regex equivalence," in *EMNLP-IJCNLP*, 2019, pp. 6426–6432.
- [7] S. RTCA, "Do-333 formal methods supplement to do-178c and do-278a," *Tech. Rep.*, 2011.
- [8] K. J. Hayhurst, *A practical tutorial on modified condition/decision coverage*. DIANE Publishing, 2001.
- [9] W. Miao, G. Pu, Y. Yao, T. Su, D. Bao, Y. Liu, S. Chen, and K. Xiong, "Automated requirements validation for atp software via specification review and testing," in *International Conference on Formal Engineering Methods*. Springer, 2016, pp. 26–40.
- [10] Z. Wang, G. Pu, J. Li, J. He, S. Qin, K. G. Larsen, J. Madsen, and B. Gu, "Mdm: A mode diagram modeling framework," *arXiv preprint arXiv:1301.0046*, 2013.
- [11] T. Parr, *The definitive ANTLR 4 reference*. Pragmatic Bookshelf, 2013.
- [12] L. De Moura and N. Bjørner, "Z3: An efficient smt solver," in *TACAS*, 2008, pp. 337–340.
- [13] D. Gelperin and B. Hetzel, "The growth of software testing," *Communications of the ACM*, vol. 31, no. 6, pp. 687–695, 1988.
- [14] G. J. Myers, T. Badgett, T. M. Thomas, and C. Sandler, *The art of software testing*. Wiley Online Library, 2004, vol. 2.
- [15] P. Ammann and J. Offutt, *Introduction to software testing*. Cambridge University Press, 2016.
- [16] P. C. Jorgensen, *Software testing: a craftsman's approach*. CRC press, 2018.
- [17] S. Shamshiri, R. Just, J. M. Rojas, G. Fraser, P. McMinn, and A. Arcuri, "Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t)," in *ASE*, 2015, pp. 201–211.
- [18] C. Pacheco and M. D. Ernst, "Randoo: feedback-directed random testing for java," in *OOPSLA*, 2007, pp. 815–816.
- [19] M. M. Almasi, H. Hemmati, G. Fraser, A. Arcuri, and J. Benefelds, "An industrial evaluation of unit test generation: Finding real faults in a financial application," in *ICSE-SEIP*, 2017, pp. 263–272.
- [20] A. Aniculaesei, F. Howar, P. Denecke, and A. Rausch, "Automated generation of requirements-based test cases for an adaptive cruise control system," in *VST*, 2018, pp. 11–15.
- [21] M. Li, B. Meng, H. Yu, K. Siu, M. Durling, D. Russell, C. McMillan, M. Smith, M. Stephens, and S. Thomson, "Requirements-based automated test generation for safety critical software," in *DASC*, 2019, pp. 1–10.
- [22] Y. Ma, Y. Huang, Z. Hu, X. Xiao, and X. Liu, "Paladin: Automated generation of reproducible test cases for android apps," in *Proceedings of the 20th International Workshop on Mobile Computing Systems and Applications*, 2019, pp. 99–104.
- [23] W. Wang, D. Li, W. Yang, Y. Cao, Z. Zhang, Y. Deng, and T. Xie, "An empirical study of android test generation tools in industrial cases," in *ASE*, 2018, pp. 738–748.
- [24] X. Bai, W. Dong, W.-T. Tsai, and Y. Chen, "WsdL-based automatic test case generation for web services testing," in *SOSE*, 2005, pp. 207–212.
- [25] Y. G. Kim, H. S. Hong, D.-H. Bae, and S. D. Cha, "Test cases generation from uml state diagrams," *IEEE Proceedings-Software*, vol. 146, no. 4, pp. 187–192, 1999.
- [26] H. Kim, S. Kang, J. Baik, and I. Ko, "Test cases generation from uml activity diagrams," in *SNPD*, vol. 3, 2007, pp. 556–561.
- [27] B. Hasling, H. Goetz, and K. Beetz, "Model based testing of system requirements using uml use case models," in *ICST*, 2008, pp. 367–376.
- [28] F. Daniel, E. Eduard, A. Wasif, S. Daniel, G. Thomas, and K. Avenir, "From natural language requirements to passive test cases using guarded assertions," in *QRS*, 2018, pp. 470–481.
- [29] C. Gao, Q. Zhang, and J. Sun, "Automatic train protection method, vehicle on-board controller and train based on vehicle-vehicle communication," May 16 2019, uS Patent App. 15/823,343.
- [30] A. Almohammad, J. F. Ferreira, A. Mendes, and P. White, "Reqcap: Hierarchical requirements modeling and test generation for industrial control systems," in *REW*, 2017, pp. 351–358.
- [31] E. Börger, "The abstract state machines method for high-level system design and analysis," in *Formal Methods: State of the Art and New Directions*. Springer, 2010, pp. 79–116.
- [32] D. Harel, "Statecharts: A visual formalism for complex systems," *Science of computer programming*, vol. 8, no. 3, pp. 231–274, 1987.
- [33] S. P. Miller, M. W. Whalen, and D. D. Cofer, "Software model checking takes off," *Communications of the ACM*, vol. 53, no. 2, pp. 58–64, 2010.
- [34] (2019) Ansys scade suite. [Online]. Available: <https://www.ansys.com/products/embedded-software/ansys-scade-suite>
- [35] (2019) Rtc's guide. [Online]. Available: <http://www.visionmc.com/>
- [36] Y. Huang, J. Feng, H. Zheng, J. Zhu, S. Wang, S. Jiang, W. Miao, and G. Pu, "Prema: a tool for precise requirements editing, modeling and analysis," in *ASE*, 2019, pp. 1166–1169.
- [37] S. Park, B. M. Hossain, I. Hussain, C. Csallner, M. Grechanik, K. Taneja, C. Fu, and Q. Xie, "Carfast: Achieving higher statement coverage faster," in *FSE*, 2012, pp. 1–11.
- [38] P. H. Feiler, D. P. Gluch, and J. J. Hudak, "The architecture analysis & design language (aadl): An introduction," Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst, Tech. Rep., 2006.
- [39] J. Feng, W. Miao, H. Zheng, Y. Huang, J. Li, Z. Wang, T. Su, B. Gu, G. Pu, M. Yang *et al.*, "Frep: an automated and formal approach to requirement modeling and analysis in aircraft control domain," in *ESEC/FSE*, 2020, pp. 1376–1386.